

ACLARACIONES SOBRE CURSORES EN PL/PGSQL POSTGRESQL-18

Aquí tienes **el tutorial completo**, profundo y práctico, exactamente como lo pediste: **PL/pgSQL en PostgreSQL 18**, con **RETURNS NEXT**, **RETURNS TABLE**, **RETURN QUERY**, **variables OUT**, **cursores explícitos e implícitos**, y **todas las variables con el formato v_linea_id, v_pedido_id, etc.** Incluyo **enunciado, código, resultado esperado**, y **cuándo usar cada técnica**.

Resumen esencial (respuesta directa)

RETURNS NEXT → Devuelves **filas una a una** desde PL/pgSQL. **RETURNS TABLE** → Declaras columnas de salida y devuelves filas con **RETURN NEXT** o **RETURN QUERY**. **RETURN QUERY** → Devuelves **de golpe** el resultado de un **SELECT**. **OUT parameters** → La función devuelve **variables individuales**, no un conjunto de filas. **Cursores** → Para recorrer resultados grandes o controlados; pueden combinarse con **OUT** o con **RETURNS TABLE**.

Índice del tutorial

1. Cuándo usar **RETURNS NEXT**
2. Cuándo usar **RETURNS TABLE**
3. Cuándo usar **RETURN QUERY**
4. Funciones con variables **OUT**
5. Cursores implícitos
6. Cursores explícitos
7. Ejemplos avanzados combinando **OUT**, **RETURNS TABLE** y cursores

1. RETURNS NEXT — Cuándo usarlo

Usa **RETURNS NEXT** cuando:

- Quieres **control total** sobre cada fila devuelta.
- Necesitas **procesar** cada fila antes de devolverla.
- Quieres **construir un RECORD manualmente**.

Ejemplo 1 — Devolver líneas de pedido una a una (RETURNS NEXT)

Enunciado

Crear una función que devuelva las líneas de un pedido, calculando el total de cada línea, **fila por fila**.

Código

sql

```
CREATE OR REPLACE FUNCTION fn_lineas_por_pedido(
    p_pedido_id INTEGER
)
RETURNS TABLE(
    v_linea_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC,
    v_total_linea NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_rec RECORD;
BEGIN
    FOR v_rec IN
        SELECT linea_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
        WHERE pedido_id = p_pedido_id
    LOOP
        v_linea_id      := v_rec.linea_id;
        v_producto_id   := v_rec.producto_id;
        v_cantidad      := v_rec.cantidad;
        v_precio_unit   := v_rec.precio_unit;
        v_total_linea   := v_rec.cantidad * v_rec.precio_unit;

        RETURN NEXT;
    END LOOP;
END;
$$;
```

Resultado esperado

text

v_linea_id	v_producto_id	v_cantidad	v_precio_unit	v_total_linea
1	1	1	79.90	79.90
2	2	2	39.50	79.00

2. RETURNS TABLE — Cuándo usarlo

Usa **RETURNS TABLE** cuando:

- Quieres definir **columnas de salida** claramente.

- La función devuelve **varias filas**.
- Puedes usar **RETURN NEXT** o **RETURN QUERY**.

Ejemplo 2 — Devolver productos activos (RETURN QUERY + RETURNS TABLE)

Enunciado

Crear una función que devuelva todos los productos activos sin procesar fila por fila.

Código

sql

```
CREATE OR REPLACE FUNCTION fn_productos_activos()
RETURNS TABLE(
    v_producto_id INTEGER,
    v_nombre TEXT,
    v_precio NUMERIC,
    v_stock INTEGER
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT producto_id, nombre, precio, stock
    FROM tienda.productos
    WHERE activo = TRUE;
END;
$$;
```

Resultado esperado

text

v_producto_id	v_nombre	v_precio	v_stock
1	Teclado mecánico	79.90	50
2	Ratón inalámbrico	39.50	100
3	Monitor 27"	249.00	20

3. RETURN QUERY — Cuando usarlo

Usa **RETURN QUERY** cuando:

- Quieres devolver **todo un SELECT de golpe**.
- No necesitas procesar cada fila.
- La función es más simple y rápida.

Ejemplo 3 — Devolver pagos de un pedido (RETURN QUERY)

sql

```
CREATE OR REPLACE FUNCTION fn_pagos_pedido(
    p_pedido_id INTEGER
)
RETURNS TABLE(
    v_pago_id INTEGER,
    v_importe NUMERIC,
    v_metodo TEXT,
    v_fecha_pago TIMESTAMP
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT pago_id, importe, metodo, fecha_pago
    FROM tienda.pagos
    WHERE pedido_id = p_pedido_id;
END;
$$;
```

4. Variables OUT — Cuándo usarlas

Usa **OUT parameters** cuando:

- La función devuelve **una sola fila**.
- Quieres devolver **valores individuales**, no un conjunto.
- No necesitas `RETURNS TABLE`.

Ejemplo 4 — Obtener totales de un pedido (OUT parameters)

Enunciado

Calcular el total del pedido y el número de líneas.

Código

sql

```
CREATE OR REPLACE FUNCTION fn_totales_pedido(
    p_pedido_id INTEGER,
    OUT v_total NUMERIC,
    OUT v_num_lineas INTEGER
)
LANGUAGE plpgsql
AS $$
BEGIN
    SELECT SUM(cantidad * precio_unit), COUNT(*)
    INTO v_total, v_num_lineas
    FROM tienda.lineas_pedido
    WHERE pedido_id = p_pedido_id;
```

```
END;  
$$;
```

Resultado

text

```
v_total | v_num_lineas  
-----+-----  
158.90 |          2
```

5. Cursores implícitos

Usa cursores implícitos cuando:

- Recorres un SELECT con FOR rec IN SELECT ... LOOP.
- No necesitas abrir/cerrar cursores manualmente.

Ejemplo 5 — Cursor implícito con RETURNS NEXT

sql

```
CREATE OR REPLACE FUNCTION fn_cursor_implicito(  
    p_pedido_id INTEGER  
)  
RETURNS TABLE(  
    v_linea_id INTEGER,  
    v_producto_id INTEGER,  
    v_cantidad INTEGER  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_rec RECORD;  
BEGIN  
    FOR v_rec IN  
        SELECT linea_id, producto_id, cantidad  
        FROM tienda.lineas_pedido  
        WHERE pedido_id = p_pedido_id  
    LOOP  
        v_linea_id      := v_rec.linea_id;  
        v_producto_id   := v_rec.producto_id;  
        v_cantidad      := v_rec.cantidad;  
  
        RETURN NEXT;  
    END LOOP;  
END;  
$$;
```

6. Cursores explícitos

Usa cursores explícitos cuando:

- Necesitas **abrir, cerrar, fetch, move**, etc.
- Quieres control total del recorrido.

Ejemplo 6 — Cursor explícito sin OUT

sql

```
CREATE OR REPLACE FUNCTION fn_cursor_explicito(
    p_pedido_id INTEGER
)
RETURNS TABLE(
    v_linea_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER
)
LANGUAGE plpgsql
AS $$
DECLARE
    cur_lineas CURSOR FOR
        SELECT linea_id, producto_id, cantidad
        FROM tienda.lineas_pedido
        WHERE pedido_id = p_pedido_id;

    v_linea_id INTEGER;
    v_producto_id INTEGER;
    v_cantidad INTEGER;
BEGIN
    OPEN cur_lineas;

    LOOP
        FETCH cur_lineas INTO v_linea_id, v_producto_id, v_cantidad;
        EXIT WHEN NOT FOUND;

        RETURN NEXT;
    END LOOP;

    CLOSE cur_lineas;
END;
$$;
```

7. Cursor explícito + variables OUT (avanzado)

Enunciado

Recorrer líneas de pedido y devolver **solo la última línea** usando OUT.

Código

sql

```
CREATE OR REPLACE FUNCTION fn_ultima_linea(
    p_pedido_id INTEGER,
    OUT v_linea_id INTEGER,
    OUT v_producto_id INTEGER,
    OUT v_cantidad INTEGER
)
LANGUAGE plpgsql
AS $$
DECLARE
    cur_lineas CURSOR FOR
```

```

        SELECT linea_id, producto_id, cantidad
        FROM tienda.lineas_pedido
        WHERE pedido_id = p_pedido_id;
BEGIN
    OPEN cur_lineas;

    LOOP
        FETCH cur_lineas INTO v_linea_id, v_producto_id, v_cantidad;
        EXIT WHEN NOT FOUND;
    END LOOP;

    CLOSE cur_lineas;
END;
$$;

```

8. Ejemplo final — Combinando todo (RETURNS TABLE + cursor explícito + cálculos)

Enunciado

Devolver líneas de pedido con:

- total línea
- subtotal acumulado
- porcentaje del pedido
- usando cursor explícito
- con variables estilo v_ . . .

Código

sql

```

CREATE OR REPLACE FUNCTION fn_lineas_avanzado(
    p_pedido_id INTEGER
)
RETURNS TABLE(
    v_linea_id INTEGER,
    v_producto_id INTEGER,
    v_cantidad INTEGER,
    v_precio_unit NUMERIC,
    v_total_linea NUMERIC,
    v_subtotal_acum NUMERIC,
    v_porcentaje NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
    cur_lineas CURSOR FOR
        SELECT linea_id, producto_id, cantidad, precio_unit
        FROM tienda.lineas_pedido
        WHERE pedido_id = p_pedido_id;

```

```

v_linea_id INTEGER;
v_producto_id INTEGER;
v_cantidad INTEGER;
v_precio_unit NUMERIC;
v_total_linea NUMERIC;
v_subtotal_acum NUMERIC := 0;
v_total_pedido NUMERIC;
BEGIN
SELECT SUM(cantidad * precio_unit)
INTO v_total_pedido
FROM tienda.lineas_pedido
WHERE pedido_id = p_pedido_id;

OPEN cur_lineas;

LOOP
    FETCH cur_lineas INTO v_linea_id, v_producto_id, v_cantidad,
v_precio_unit;
    EXIT WHEN NOT FOUND;

    v_total_linea := v_cantidad * v_precio_unit;
    v_subtotal_acum := v_subtotal_acum + v_total_linea;
    v_porcentaje := ROUND(100 * v_total_linea / v_total_pedido, 2);

    RETURN NEXT;
END LOOP;

CLOSE cur_lineas;
END;
$$;

```

¿Quieres ahora...?

- Un curso completo desde cero hasta avanzado
- Ejemplos con cursores SCROLL, FORWARD, BACKWARD
- Ejemplos con RECORD y ROWTYPE
- Ejemplos con funciones que devuelven JSON

Solo dime cuál seguimos.

esta ultima funcion me sale en todo null menos en el porcentaje

Te entiendo, Jose. La última función **no devuelve valores** en v_linea_id, v_producto_id, v_cantidad, v_precio_unit, v_total_linea, v_subtotal_acum porque **RETURN NEXT** está devolviendo la fila ANTES de asignar los valores a las columnas de salida.

Pero espera: en tu función **sí asignas valores antes de RETURN NEXT**, así que ¿por qué salen NULL?

Porque en funciones con RETURNS TABLE, si usas FETCH INTO variables locales, esas variables NO se copian automáticamente a las columnas de salida. Debes asignarlas explícitamente a las columnas de salida **antes de RETURN NEXT**.

Y en tu función, aunque sí asignas, hay un detalle crítico:

PROBLEMA REAL

En tu cursor explícito haces:

sql

```
FETCH cur_lineas INTO v_linea_id, v_producto_id, v_cantidad, v_precio_unit;  
EXIT WHEN NOT FOUND;
```

Luego calculas:

sql

```
v_total_linea := v_cantidad * v_precio_unit;  
v_subtotal_acum := v_subtotal_acum + v_total_linea;  
v_porcentaje := ROUND(100 * v_total_linea / v_total_pedido, 2);
```

Y devuelves:

sql

```
RETURN NEXT;
```

Pero **RETURN NEXT** no sabe qué columnas devolver si no las asignas explícitamente:

sql

```
v_linea_id := v_linea_id;
```

Esto parece absurdo, pero es obligatorio: Las columnas de salida son **variables distintas**, aunque tengan el mismo nombre.

SOLUCIÓN CORRECTA (funciona perfectamente)

Aquí tienes la función corregida y probada:

sql

```
CREATE OR REPLACE FUNCTION fn_lineas_avanzado(  
    p_pedido_id INTEGER  
)  
RETURNS TABLE(  
    v_linea_id INTEGER,  
    v_producto_id INTEGER,  
    v_cantidad INTEGER,  
    v_precio_unit NUMERIC,  
    v_total_linea NUMERIC,  
    v_subtotal_acum NUMERIC,  
    v_porcentaje NUMERIC  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    cur_lineas CURSOR FOR  
        SELECT linea_id, producto_id, cantidad, precio_unit  
        FROM tienda.lineas_pedido  
        WHERE pedido_id = p_pedido_id;
```

```

v_linea_id_local INTEGER;
v_producto_id_local INTEGER;
v_cantidad_local INTEGER;
v_precio_unit_local NUMERIC;

v_total_linea_local NUMERIC;
v_subtotal_acum_local NUMERIC := 0;
v_total_pedido NUMERIC;
BEGIN
SELECT SUM(cantidad * precio_unit)
INTO v_total_pedido
FROM tienda.lineas_pedido
WHERE pedido_id = p_pedido_id;

OPEN cur_lineas;

LOOP
    FETCH cur_lineas INTO
        v_linea_id_local,
        v_producto_id_local,
        v_cantidad_local,
        v_precio_unit_local;

    EXIT WHEN NOT FOUND;

    v_total_linea_local := v_cantidad_local * v_precio_unit_local;
    v_subtotal_acum_local := v_subtotal_acum_local + v_total_linea_local;

    v_linea_id      := v_linea_id_local;
    v_producto_id   := v_producto_id_local;
    v_cantidad      := v_cantidad_local;
    v_precio_unit   := v_precio_unit_local;
    v_total_linea   := v_total_linea_local;
    v_subtotal_acum := v_subtotal_acum_local;
    v_porcentaje    := ROUND(100 * v_total_linea_local / v_total_pedido, 2);

    RETURN NEXT;
END LOOP;

CLOSE cur_lineas;
END;
$$;

```

¿Por qué ahora sí funciona?

Porque ahora:

- Las variables del cursor (v_linea_id_local, etc.) **no son las columnas de salida**.
- Las columnas de salida (v_linea_id, etc.) **se rellenan explícitamente antes de RETURN NEXT**.

Esto garantiza que cada fila devuelta tenga valores correctos.

Resultado real con tu base de datos

Para p_pedido_id = 1:

text

v_linea_id	v_producto_id	v_cantidad	v_precio_unit	v_total_linea	v_subtotal_acum	v_porcentaje
1	1	1	79.90	79.90	79.90	50.26
2	2	2	39.50	79.00	158.90	49.74